

Marking Rubric for Lab #1 Assessment (Week 4)

FIT3143 Parallel Computing © Monash University 2026

LAB1 Marking Rubric

		HD	D	C	P	N
Methodology (20%)						
Task 1 Serial Code	Code Correctness, Style, and Clarity (4%)	The program is capable of printing/outputting <u>a sorted list</u> of prime numbers that are strictly less than an integer n, provided by the user. The code has performed significant optimisation to reduce and eliminate unnecessary computations. The coding style follows industry (level) practices: - Good programming style (and) - Code is well-commented with industry level documentation.	The program is capable to printing/outputting <u>a sorted list</u> of prime numbers which are strictly less than an integer n, provided by the user. The code has performed significant optimisation to reduce and eliminate unnecessary computations. The code is readable and follows good programming practice with sufficient comments	The program is capable to find and printing/outputting <u>a sorted list</u> of prime numbers which are strictly less than an integer n, provided by the user.	The program is capable to find and printing/outputting prime numbers which are strictly less than an integer n, provided by the user.	The code contains major flaws or fails to find prime numbers (or) The program print/output values that are not prime numbers (or) The program fails to be properly executed due to major bugs/flaws
Task 2 POSIX Threads	Code Correctness and Clarity (8%)	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with POSIX Threads which are strictly less than an integer n, provided by the user. The code has performed significant optimisation to reduce and eliminate unnecessary computations and bottlenecks to improve parallel speedup.	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with POSIX Threads which are strictly less than an integer n, provided by the user. The overall computation speedup is strictly larger than 1. The code has performed significant optimisation to reduce and eliminate	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with POSIX Threads which are strictly less than an integer n, provided by the user. The overall computation speedup is strictly larger than 1.	The program is capable to find and print/output prime numbers with POSIX Threads which are strictly less than an integer n, provided by the user.	The parallel code contains major flaws affecting the correctness of the program (or) The code completely fails to perform parallelism using POSIX Threads (or) The code fails to find prime numbers (or) The program print/output

		The overall computation speedup is strictly larger than 1 and <u>close to achieving a linear speed up</u> (i.e. speed up close to the number of computing cores available) The coding style follows industry (level) practices: - Good programming style (and) - Code is well-commented with industry level documentation.	unnecessary computations and bottlenecks to improve parallel speedup. The code is readable and follows good programming practice with sufficient comments			values that are not prime numbers (or) The program fails to be properly executed due to major bugs/flaws (including deadlock and system crash)
Task 3 Open MP	Code Correctness and Clarity (8%)	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with Open MP which are strictly less than an integer n, provided by the user. The code has performed significant optimisation with different Open MP features to reduce and eliminate unnecessary computations and bottlenecks to improve parallel speedup. The overall computation speedup is strictly larger than 1 and <u>close to achieving a linear speed up</u> (i.e. speed up close to the number of computing cores available) The coding style follows industry (level) practices: - Good programming style (and)	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with Open MP which are strictly less than an integer n, provided by the user. The overall computation speedup is strictly larger than 1. The code has performed significant optimisation with different Open MP features to reduce and eliminate unnecessary computations and bottlenecks to improve parallel speedup. The code is readable and follows good programming practice with sufficient comments	The program is capable to find and print/output <u>a sorted list</u> of prime numbers with Open MP which are strictly less than an integer n, provided by the user. The overall computation speedup is strictly larger than 1.	The program is capable to find and print/output prime numbers with Open MP which are strictly less than an integer n, provided by the user.	The parallel code contains major flaws affecting the correctness of the program (or) The code completely fails to perform parallelism using Open MP (or) The code fails to find prime numbers (or) The program print/output values that are not prime numbers (or) The program fails to be properly executed due to major bugs/flaws (including deadlock and system crash)

		- Code is well-commented with industry level documentation.				
Presentation Quality and Q&A (80%)						
Presentation Quality (Clarity and Organisation) (10%)		The presentation is between 6 to 8 minutes. The presentation is <u>exceptionally</u> clear, well-organised, and easy to follow. The conclusion section summarises the presentation well, and discusses major issues, limitations, and future work. The presentation is concise, precise, and accurate (i.e., materials have been digested, analyzed, and chosen carefully without unnecessary information or details). Students are required to extensively use parallel computing terminologies (instead of general computing terms) at this level.	The presentation is between 6 to 8 minutes. The presentation is clear and organised with minor areas for improvement. The conclusion section summarises the presentation, and discusses major issues, limitations, and future work.	The presentation is generally understandable but has major rooms for improvements.	The presentation is somewhat understandable but lacks either organisation or content flow.	The presentation is largely unclear, poorly organised, and hard to follow.
Content Coverage (30%)	Task 1 (6%)	The coding approach for Task 1 has been clearly presented with no clear omissions. Sample output from the implemented program has been presented. Improvements and	The coding approach for Task 1 has been clearly presented. Sample output from the implemented program has been presented. Improvements and optimisations made to the	The coding approach for Task 1 has been clearly presented. Sample output from the implemented program has been presented.	The coding approach for Task 1 has been clearly presented.	Presentation for Task 1 (serial code) is missing. or The approach presented has no relation with Task 1.

		optimisations made to code to reduce and eliminate unnecessary computations has also been clearly presented. Statistically significant experimental results to showcase the improvements have also been presented.	code to reduce and eliminate unnecessary computations has also been clearly presented.			
	Task 2 (12%)	The coding approach for Task 2 has been clearly presented with no clear omissions. The parallel partitioning scheme and workload distribution tasks have also been presented. All of the required graphs in the specification are presented, with no flaws/data omissions.	The coding approach for Task 2 has been clearly presented. The parallel partitioning scheme and workload distribution tasks have also been presented. All of the required graphs in the specification are presented, with no major flaws and no major data omissions.	The coding approach for Task 2 has been clearly presented. The parallel partitioning scheme and workload distribution tasks have also been presented. Some of the required graphs in the specification are presented.	The coding approach for Task 2 has been clearly presented.	Presentation for Task 2 (serial code) is missing. or The approach presented has no relation with Task 2.
	Task 3 (12%)	The coding approach for Task 3 has been clearly presented with no clear omissions. The parallel partitioning scheme and workload distribution tasks have also been presented. All of the required graphs in the specification are presented, with no flaws/data omissions.	The coding approach for Task 3 has been clearly presented. The parallel partitioning scheme and workload distribution tasks have also been presented. All of the required graphs in the specification are presented, with no major flaws and no major data omissions.	The coding approach for Task 3 has been clearly presented. The parallel partitioning scheme and workload distribution tasks have also been presented. Some of the required graphs in the specification are presented.	The coding approach for Task 3 has been clearly presented.	Presentation for Task 3 (serial code) is missing. or The approach presented has no relation with Task 3.
Q&A (40%)		Capable of understanding and answering all the questions without external tools. All the answers are correct without any flaws.	Capable of understanding all the questions and answering all the questions without external tools. The answers are all correct without any flaws.	Capable of understanding all the questions and answering all the questions without external tools. The answers are largely correct, despite some minor	Capable of understanding all the questions and answering all the questions without external tools. The answers are largely correct, despite some major	Fails to understand the questions (or) Relies on external tools (e.g., AI, software tools, or online tools) to understand the

	The answers are concise, precise, and accurate (i.e., fully answer the question without unnecessary information or details). Students are required to answer by using parallel computing terminologies (instead of general computing terms) at this level.		flaws.	flaws.	questions or answer the questions. (or) Answers are completely wrong with major flaws/issues.
--	--	--	--------	--------	---

Special Consideration & Extensions

- From the special consideration procedure, short extensions are not available for group assessments, in-class assessments (including in-class tests/quizzes and presentations), practical assessments (including laboratories), mid-semester tests, or scheduled final assessments. For these assessment types, students must instead apply for special consideration.
- Unless a special consideration extension is approved, the student group/team will generally receive a 0 mark if they fail to attend the in-class session, fail to demonstrate the work, or fail to submit the required files to Moodle.
- For students with special considerations approved by the special consideration team, an alternative form of assessment must be submitted no later than the newly approved deadline. Students are reminded to read and follow the special consideration and alternate assessment sections in the assessment specification.
- If the extended due date falls after Week 14 and you plan to submit on/after Week 14, students are advised to contact the Chief Examiner and/or the Unit Coordinator, as grades may need to be withheld to allow extra time for marking.
- Students who have received special considerations but failed to submit the required files to Moodle by the newly approved deadline for their alternative assessment will be treated as having made no submission (or as having submitted late, if the file eventually arrives after the newly approved due date). For alternative assessments, late penalties will still apply for late submissions.